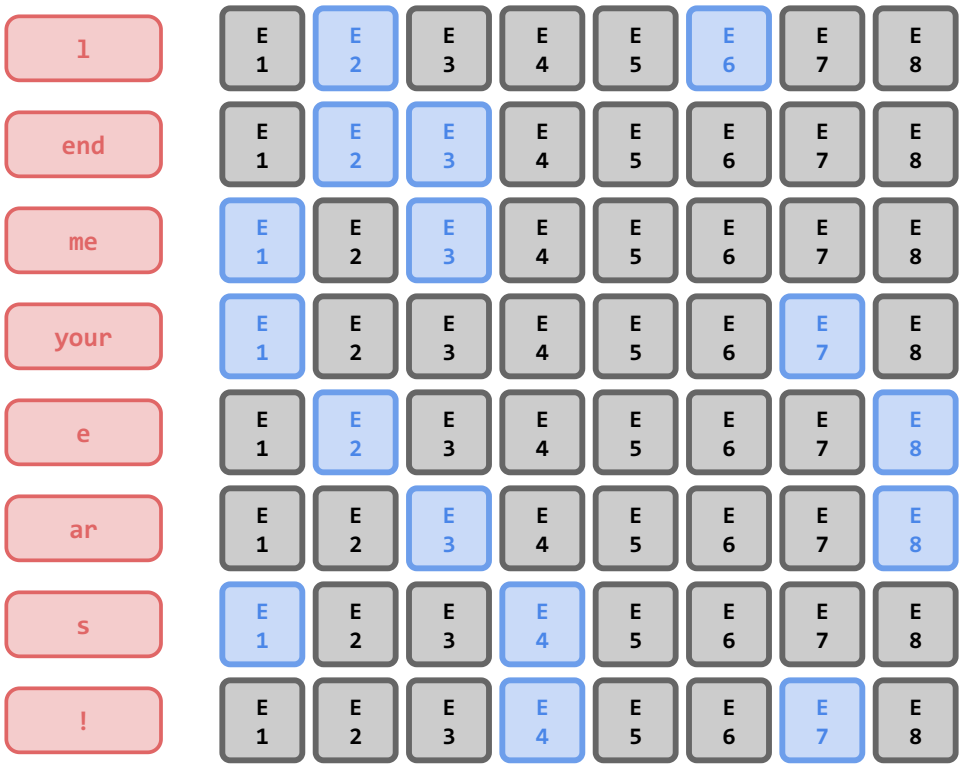


UNIT 01 · CHAPTER 11

Mixture of Experts

CSE 10124 · Building ChatGPT
William Theisen · University of Notre Dame

Counting Parameters

By the end of the last chapter attention had stopped being the expensive part of a block, and none of its tricks touched the other half, the **feed-forward network** (FFN). So where are a model's weights? Counting them is just multiplying shapes, here at irishQwen's default (Qwen3-0.6B) shapes:

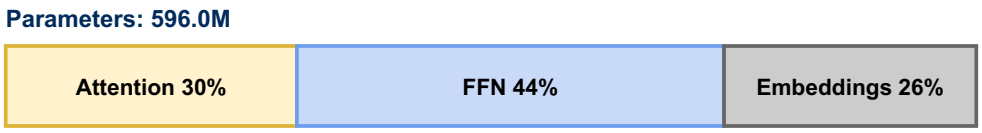
```python
# irishQwen's default shapes (Qwen3-0.6B)
V, d, layers = 151936, 1024, 28
q_dim, kv_dim, ffn = 16 * 128, 8 * 128, 3072

attn = d * q_dim + 2 * d * kv_dim + q_dim * d
ffn_params = 3 * d * ffn          # gate, up and down
embed = V * d                    # shared with LM head

total = layers * (attn + ffn_params) + embed
for name, n in [('attention', layers * attn),
                ('FFN', layers * ffn_params),
                ('embeddings', embed)]:
    print(f'{name:>10}: {n/1e6:6.1f}M  {n/total:.0%}')
print(f'{"total":>10}: {total/1e6:6.1f}M')
```

OUTPUT

```
 attention:  176.2M  30%
       FFN:  264.2M  44%
embeddings:  155.6M  26%
     total:  596.0M
```



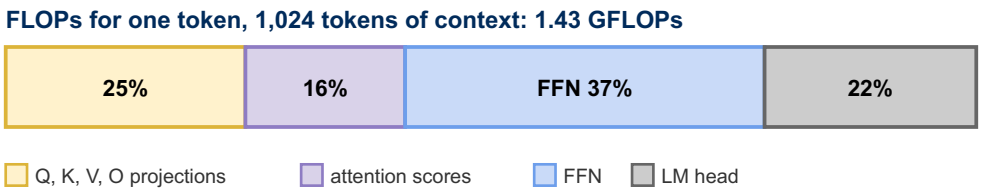
28 blocks of attention and FFN, one embedding table shared with the LM head

The FFNs hold **44%** of the model, **more than attention and more than the embedding table**.

Counting Compute

Memory is one cost, the other is the work it takes to produce each token. A weight that gets used costs one multiply and one add, two **FLOPs** (floating point operations), so every token that goes through a block costs about twice the block's weights. Attention also has to compare the new token's query with every cached key and mix every cached value, which grows with the context.

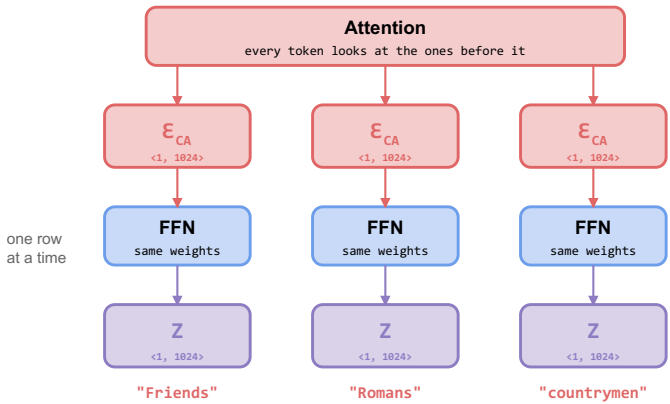
For one new token with `1,024` tokens of context, irishQwen at Qwen3-0.6B's shapes does `352.3` million FLOPs in the attention projections, `234.9` million comparing with the cache, `528.5` million in the FFNs and `311.2` million in the LM head that turns the last vector into scores for all `151,936` tokens:



The FFN is again the biggest piece, `37%` of the work. Worse, it's the piece we'd most like to grow. **Every token goes through every FFN weight**, so a model with twice the FFN costs twice as much for every token it ever reads or writes.

The Feed-Forward Network

So what is all that FFN for? Attention is the only place where tokens talk to each other. Each token comes out of it as a context-aware embedding $\mathcal{E}_{CA}$, and the FFN then works on **each token's row on its own**, with the same weights for every token:



Pictured above are the first words of Antony's speech in *Julius Caesar*, our example for the rest of the chapter.

It turns out the FFN is where a lot of what a model "knows" seems to live. Geva and colleagues (2021) found that each FFN hidden unit acts like a small memory, switching on for certain patterns in the text and pushing the prediction toward the words that tend to follow. Later studies changed single facts a model recalls by editing FFN weights (Meng and colleagues, 2022). Nobody can point at the one unit that knows Brutus stabbed Caesar, but this is why bigger models mostly mean bigger FFNs.

SwiGLU

irishQwen's FFN is a **SwiGLU** (the class in `swiglu.py`), the same one Qwen3, Llama and Mistral use. It has three weight matrices and no biases. Two of them take the token up to the hidden size, one through a SiLU (the "gate") and one left alone (the "up"), and the two are multiplied number by number. The third, W_{down}, brings the result back down (the shapes below are those of the smaller model we use from here on):





gate: how much
up: what to let through

The gate is what makes it more than two linear layers. SiLU of a very negative number is about zero, so for each hidden unit the gate branch decides **how much of the up branch gets through**.

From here on we'll use the class's Shakespeare model, which keeps Qwen3's design at a size we can run in the browser: `256` numbers per token and `768` hidden units. Our weights are random (initialized the way irishQwen's `LinearLayer` does it), and like `LinearLayer` each matrix is stored as (out, in), hence the `.T` in the code on the next page.

SwiGLU in Code

Here is irishQwen's SwiGLU for one token of the Shakespeare model:

```python
import torch
import torch.nn.functional as F

torch.manual_seed(0)
torch.set_printoptions(precision=2)
d_model, ffn_dim = 256, 768     # irishQwen on Shakespeare

def init(n_out, n_in):          # irishQwen's LinearLayer
    return torch.randn(n_out, n_in) * (2 / n_in) ** 0.5

W_gate = init(ffn_dim, d_model)
W_up = init(ffn_dim, d_model)
W_down = init(d_model, ffn_dim)

def swiglu(x):
    h = F.silu(x @ W_gate.T) * (x @ W_up.T)
    return h @ W_down.T

x = torch.randn(1, d_model)     # one token's E_CA
z = swiglu(x)
print('x:', tuple(x.shape), ' z:', tuple(z.shape))
n = W_gate.numel() + W_up.numel() + W_down.numel()
print('parameters:', n)
```

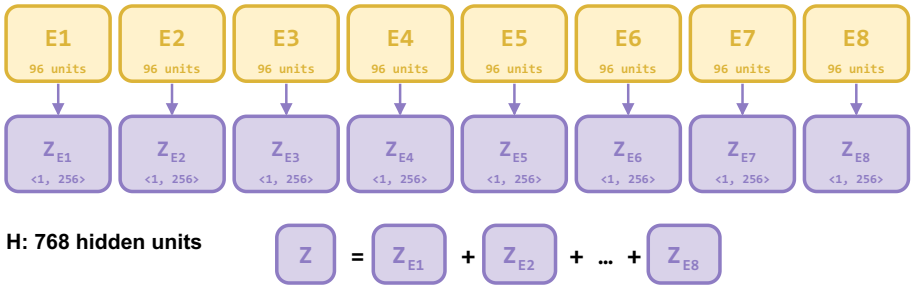
OUTPUT

```
x: (1, 256)  z: (1, 256)
parameters: 589824
```

One token in, one token out, the same shape, and **589,824** weights used along the way. That's one block's FFN, for one token, in a small model.

Experts

Each hidden unit uses only its own row of the gate and up weights and its own column of W_{down}, and never looks at another unit. So we can cut them into groups, run each group as a small SwiGLU, and add the results:




```
n_experts = 8
size = ffn_dim // n_experts     # 96 hidden units each

def expert(i, x):
    rows = slice(i * size, (i + 1) * size)
    h = F.silu(x @ W_gate[rows].T) * (x @ W_up[rows].T)
    return h @ W_down[:, rows].T

z_all = sum(expert(i, x) for i in range(n_experts))
print('size of each expert:', size)
print('biggest difference:', (z_all - z).abs().max())
```

OUTPUT

```
size of each expert: 96
biggest difference: tensor(1.13e-06)
```

The eight pieces add up to the FFN's answer (the `0.00000113` is rounding). Each piece is a complete feed-forward network of its own, just narrower, with `96` hidden units instead of `768`. We call each one an **expert**.

Sparse Activation

The saving comes from not running all of them. If each token only goes through k of the eight experts, it only uses those experts' weights:

```
k = 2                              # experts used per token
per_expert = 3 * d_model * size
print('per expert:', per_expert)
print('total:     ', n_experts * per_expert)
print('active:    ', k * per_expert)
```

OUTPUT

```
per expert: 73728
total:      589824
active:     147456
```



With two experts per token the layer still has all `589,824` weights, but **each token only uses a quarter of them**. A layer where every weight is used for every token is **dense**, one where each token uses only a few pieces is **sparse**, and a sparse FFN built from experts is a **Mixture of Experts** (MoE).

We can't do this to an FFN that was trained dense, skip six of its eight groups and the output is just wrong. An MoE is trained this way from the start, so each expert learns to be useful on its own. Which leaves the question of which experts each token gets.

The Router

The choice is made by one more small linear layer, the **router**. Its weights W_R turn a token's $\mathcal{E}_{CA}$ into one score per expert, $S = \mathcal{E}_{CA} @ W_R$, and a higher score means "send me here". For our model W_R is `256` by `8`, only `2,048` weights.

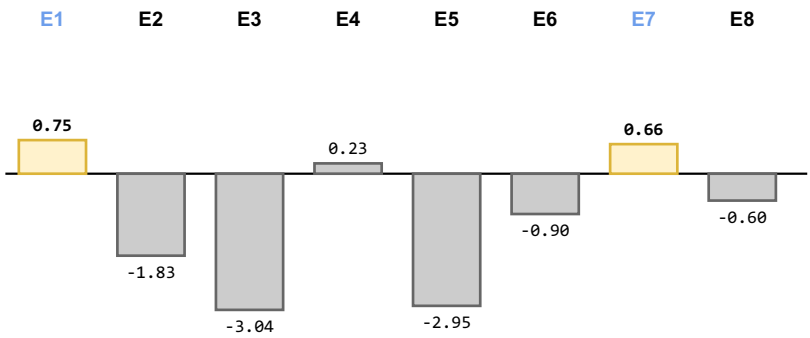
With no trained attention layer, a random row stands in for each token's $\mathcal{E}_{CA}$ (one per token in the `512`-token Shakespeare vocabulary). Token `350` is " your":

```
E = torch.randn(512, d_model)   # one row per token
W_R = init(n_experts, d_model)  # the router

x = E[[350]]                    # token 350 is " your"
s = x @ W_R.T                   # one score per expert
print(s)
```

OUTPUT

```
tensor([[ 0.75, -1.83, -3.04,  0.23, -2.95, -0.90,  0.66, -0.60]])
```



Experts `1` and `7` score highest. With random weights these picks mean nothing yet (the router hasn't learned anything about " your"), but the mechanics are exactly those of a trained model.

Top-k Gating

We keep the k largest scores with `topk` and turn just those into weights that add to one with a softmax. These are the **gates** g, how much each chosen expert counts:

$$g = \mathrm{softmax}(\mathrm{top}_k(S))$$

```
top_s, top = torch.topk(s, k)
g = torch.softmax(top_s, dim=-1)
print('experts:', top + 1)      # E1 ... E8
print('gates:', g)

p = torch.softmax(s, dim=-1)    # all 8, then rescale
print('same gates:', p[0, top] / p[0, top].sum())
```

OUTPUT

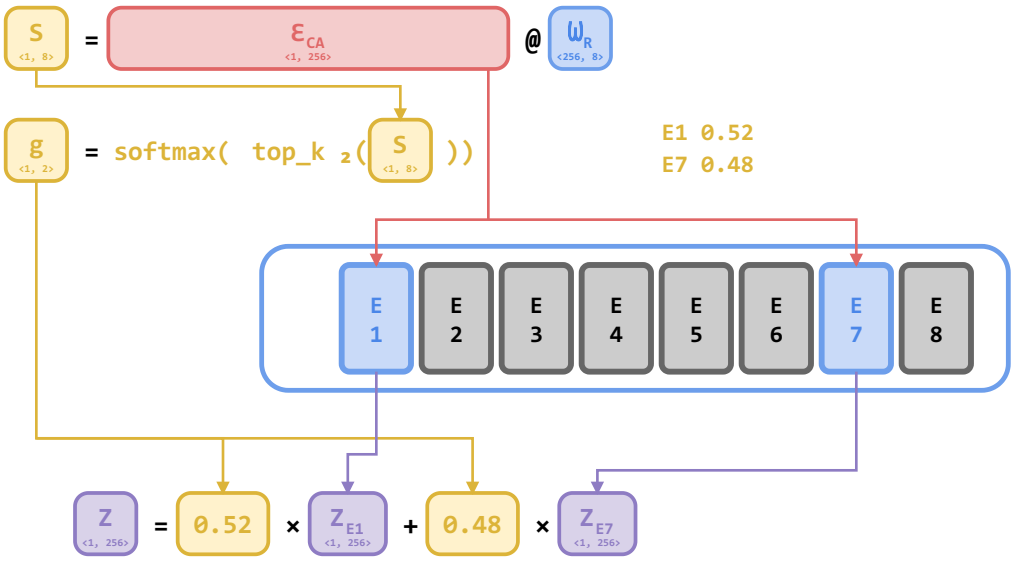
```
experts: tensor([[1, 7]])
gates: tensor([[0.52, 0.48]])
same gates: tensor([[0.52, 0.48]])
```

" your" goes to experts `1` and `7` with gates `0.52` and `0.48`. The scores were close (`0.75` and `0.66`), so the gates are close too. The last line checks that this is the same as a softmax over all eight experts, then keeping the top two and rescaling them to add to one. Some models use one form and some the other, they give the same gates.

Picking k is a design choice, like the number of attention heads. Mixtral uses two of eight, so we will too.

Mixing the Experts

Only the chosen experts run, and their outputs are added weighted by their gates, $Z = 0.52\ Z_{E1} + 0.48\ Z_{E7}$:



The router decides which experts get ε_{CA} at all, and the gates decide how much each one counts in the sum. The six experts that weren't picked do no work for this token.

The MoE Layer

In code, the router, the gates and the weighted sum fit in one short function, and the loop only ever calls the k chosen experts:

```python
def moe(x):
    s = x @ W_R.T
    top_s, top = torch.topk(s, k)
    g = torch.softmax(top_s, dim=-1)
    z = torch.zeros(1, d_model)
    for j in range(k):
        i = top[0, j].item()
        z = z + g[0, j] * expert(i, x)
    return z

z = moe(x)
print('z:', tuple(z.shape))
print(z[0, :4])
```

OUTPUT

```
z: (1, 256)
tensor([ 0.60, -0.34,  0.40,  0.01])
```

That's the whole **MoE layer**, a drop-in for the FFN with the same shapes. The gates also let training adjust the router.

Routing a Line

Every token makes its own choice. Here is the router on all **25** tokens of "Friends, Romans, countrymen, lend me your ears!" (irishQwen's Shakespeare tokenizer splits "Friends" into "F", "ri", "end", "s"):

	E1	E2	E3	E4	E5	E6	E7	E8	gates
"F"	0.02	0.09	**0.26**	0.04	0.15	**0.21**	0.18	0.05	E3 0.56 · E6 0.44
"ri"	0.05	0.01	0.20	0.03	0.04	0.07	**0.25**	**0.35**	E8 0.59 · E7 0.41
"end"	0.02	**0.47**	**0.25**	0.07	0.00	0.03	0.01	0.16	E2 0.66 · E3 0.34
"s"	**0.20**	0.01	0.06	**0.33**	0.14	0.09	0.08	0.09	E4 0.63 · E1 0.37
","	0.02	0.14	**0.19**	0.05	**0.51**	0.01	0.04	0.04	E5 0.73 · E3 0.27
" R"	0.03	0.00	0.01	0.02	0.02	**0.10**	**0.82**	0.00	E7 0.90 · E6 0.10
"om"	0.09	**0.54**	0.02	0.09	0.01	0.09	0.04	**0.12**	E2 0.82 · E8 0.18
"an"	0.02	0.00	0.00	0.09	0.03	0.05	**0.68**	**0.11**	E7 0.86 · E8 0.14
"s"	**0.20**	0.01	0.06	**0.33**	0.14	0.09	0.08	0.09	E4 0.63 · E1 0.37
","	0.02	0.14	**0.19**	0.05	**0.51**	0.01	0.04	0.04	E5 0.73 · E3 0.27
" c"	**0.25**	0.06	0.12	0.00	0.17	0.04	0.12	**0.24**	E1 0.52 · E8 0.48
"ou"	**0.28**	**0.21**	0.01	0.12	0.20	0.09	0.07	0.02	E1 0.57 · E2 0.43
"nt"	0.17	**0.18**	0.01	0.06	0.01	0.13	0.02	**0.42**	E8 0.71 · E2 0.29
"ry"	0.01	0.05	**0.53**	0.04	0.04	0.01	0.09	**0.22**	E3 0.70 · E8 0.30
"m"	0.01	0.04	**0.34**	0.03	**0.34**	0.00	0.15	0.09	E3 0.50 · E5 0.50
"en"	0.03	0.14	0.12	0.04	0.04	**0.31**	**0.18**	0.13	E6 0.63 · E7 0.37
","	0.02	0.14	**0.19**	0.05	**0.51**	0.01	0.04	0.04	E5 0.73 · E3 0.27
" l"	0.03	**0.48**	0.13	0.03	0.01	**0.19**	0.11	0.02	E2 0.72 · E6 0.28
"end"	0.02	**0.47**	**0.25**	0.07	0.00	0.03	0.01	0.16	E2 0.66 · E3 0.34
" me"	**0.42**	0.05	**0.28**	0.05	0.01	0.01	0.10	0.08	E1 0.60 · E3 0.40
" your"	**0.32**	0.02	0.01	0.19	0.01	0.06	**0.30**	0.08	E1 0.52 · E7 0.48
" e"	0.00	**0.27**	0.16	0.02	0.12	0.10	0.01	**0.32**	E8 0.54 · E2 0.46
"ar"	0.14	0.08	**0.26**	0.03	0.02	0.12	0.04	**0.31**	E8 0.54 · E3 0.46
"s"	**0.20**	0.01	0.06	**0.33**	0.14	0.09	0.08	0.09	E4 0.63 · E1 0.37
"!"	0.05	0.06	0.13	**0.16**	0.06	0.08	**0.41**	0.05	E7 0.72 · E4 0.28

Notice that "s" and "," always go to the same experts. Our stand-in rows are the same every time a token appears, whereas in a real model $\mathcal{E}_{CA}$ has been through attention and "s" in "Friends" and "s" in "ears" would look different.

Load Balancing

Adding up the picks for the whole line, `25` tokens with `2` experts each, gives `50` jobs to share between `8` experts:

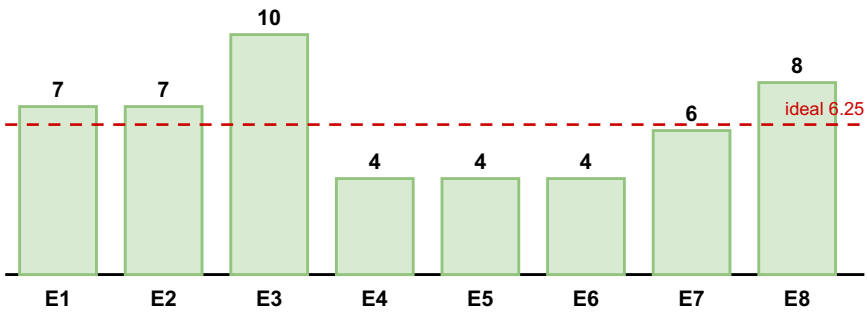
```
# "Friends, Romans, countrymen, lend me your ears!"
ids = [70, 359, 460, 115, 44, 509, 305, 300, 115,
       44, 283, 262, 414, 498, 109, 287, 44, 284,
       460, 320, 350, 335, 290, 115, 33]

S = E[ids] @ W_R.T              # <25, 8>
top_s, top = torch.topk(S, k)
G = torch.softmax(top_s, dim=-1)

counts = F.one_hot(top.flatten(), n_experts).sum(0)
print('tokens per expert:', counts)
print('ideal:', len(ids) * k / n_experts)
```

OUTPUT

```
tokens per expert: tensor([ 7,  7, 10,  4,  4,  4,  6,  8])
ideal: 6.25
```



Perfectly even would be `6.25` each. Expert `3` got `10` and three experts got `4`. That matters because big MoE models put different experts on different GPUs, so **one busy expert keeps the others waiting**, and an expert that rarely gets a token rarely gets trained.

Routing Collapse

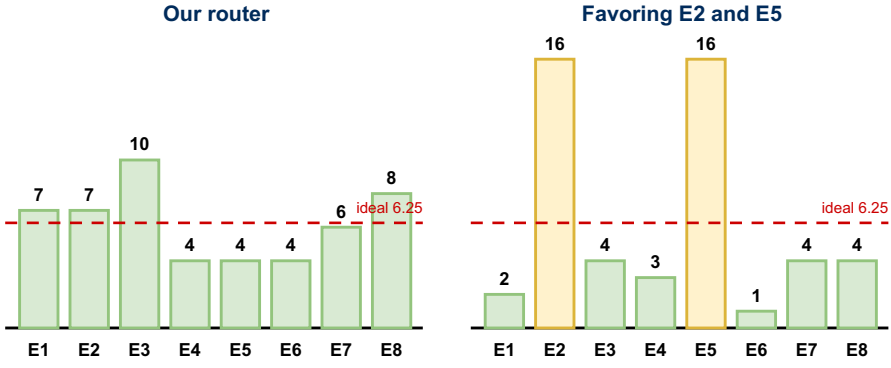
Training makes it worse. An expert that gets picked a bit more gets a bit more practice, gets a bit better, and gets picked a bit more again, until a handful of experts do everything and the rest are dead weight. This is **routing collapse**, and it has been a problem since the first sparse MoE papers.

We can fake the start of one by adding `2` to every token's scores for experts `2` and `5`, a router that has started to play favorites:

```
favor = torch.zeros(n_experts)
favor[1] = favor[4] = 2.0       # favor E2 and E5
top_f = torch.topk(S + favor, k)[1]
counts_f = F.one_hot(top_f.flatten(), n_experts).sum(0)
print('ours:     ', counts)
print('favoring: ', counts_f)
```

OUTPUT

```
ours:      tensor([ 7,  7, 10,  4,  4,  4,  6,  8])
favoring:  tensor([ 2, 16,  4,  3, 16,  1,  4,  4])
```



The two favorites now take `32` of the `50` jobs, and expert `6` gets one.

The Load-Balancing Loss

The usual fix is a second loss that is lowest when the work is shared evenly. For each expert we take f_i, its share of the picks, and P_i, its average softmax probability over the tokens:

$$L_{balance} = N \sum_{i=1}^{N} f_i \, P_i$$

```python
def balance(S):
    top = torch.topk(S, k)[1]
    counts = F.one_hot(top.flatten(), n_experts).sum(0)
    f = counts / counts.sum()   # share of the picks
    P = torch.softmax(S, dim=-1).mean(dim=0)
    return counts, n_experts * (f * P).sum()

print('ours:     ', balance(S)[1])
print('favoring: ', balance(S + favor)[1])
```

OUTPUT

```
ours:      tensor(1.04)
favoring:  tensor(1.80)
```

If every expert got `1/8` of both, the N out front makes the loss exactly `1`. The loss grows when the experts with the most picks also get the most probability, **which is what collapse looks like**: our router scores `1.04`, the favoring one `1.80`.

This form is from Google's Switch Transformer (2021). It's added to the next-token loss, scaled down (they used `0.01`) so it nudges the router without drowning out what the model is trying to learn.

Expert Capacity

Not every model leans on that loss (DeepSeek-V3 keeps only a very small one and mostly gives each expert a bias used only for picking, lowered after each step if the expert was busy and raised if it was idle). Either way, these only nudge, so a single batch can still be lopsided. Many models also give each expert a **capacity**, the most tokens it will take, usually a little above an even share (here `1.25` times it):

```
import math
cap = math.ceil(1.25 * len(ids) * k / n_experts)
print('capacity:', cap)
for scores in [S, S + favor]:
    counts, _ = balance(scores)
    print('dropped:', (counts - cap).clamp(min=0))
```

OUTPUT

```
capacity: 8
dropped: tensor([0, 0, 2, 0, 0, 0, 0, 0])
dropped: tensor([0, 8, 0, 0, 8, 0, 0, 0])
```

Each expert takes at most `8` of the line's tokens. Our router overfills expert `3` by `2`, the favoring router overfills experts `2` and `5` by `8` each. Those overflow tokens are **dropped**: they skip that expert. A dropped token isn't lost, it just comes out of this layer without that expert's contribution, carried around the layer by a connection we'll meet in the next chapter.

So capacity limits how long the other GPUs wait, at the price of sometimes skipping work.

What Experts Learn

The name suggests experts on topics, a football expert and a Shakespeare expert, with the router sending each question to the right one. That's a nice intuition, unfortunately it isn't what happens.

When Mistral looked at which experts Mixtral picked on different kinds of text (arXiv papers, biology, philosophy, maths, code), they found **no obvious pattern by topic**. The patterns they did find were about form: the word "self" in Python code tended to go to the same expert, so did indentation, and consecutive tokens often shared an expert. In their words the routing looked "more aligned with the syntax rather than the domain".

That fits how the router works. It sees one token's ε_{CA} at a time, in one layer, and it's trained only to make the next-token loss go down while keeping the experts busy. Nothing asks it to sort by subject. The experts specialize in whatever helps, and mostly that's kinds of tokens.

Total and Active Parameters

MoE models are described by two numbers. Mixtral 8x7B has about 47 billion parameters in total but uses only about 13 billion for each token (eight experts per layer, two per token, plus attention and embeddings, which every token uses). The name is a little misleading, 8 × 7 would be 56, but the eight experts share everything except the FFN.

Each token costs about as much as a 13-billion-parameter dense model, with the knowledge of something much bigger. Unfortunately, **the memory bill is for the total**. Any expert might be picked for the next token, so all 47 billion have to be loaded, and that's why running MoE models at home is harder than their active count suggests.

So an MoE is a trade. It makes compute cheap and leaves memory as expensive as ever, which is the right trade for companies serving millions of tokens a second on machines with plenty of memory.

Shared Experts

Since Mixtral, the experts in big models have gotten **smaller and more numerous**. DeepSeek's MoE design cut each expert into several narrower ones and picked more of them per token, so instead of two of eight a token might use eight of 256. The total and active sizes stay about the same, but the router has far more combinations to choose from, and each expert can specialize more narrowly.

DeepSeek also added a **shared expert** that every token goes through, with no routing at all. Whatever every token needs gets learned once there, instead of separately by every routed expert. DeepSeek-V3 has 1 shared expert and 256 routed ones per layer, with 8 routed experts per token.

Not everyone agrees it's needed. Qwen3's MoE models have 128 experts and pick 8, and the Qwen team dropped the shared experts that Qwen2.5-MoE had.

Mixture of Experts in Today's Models

The idea is old (Jacobs, Jordan, Nowlan and Hinton's "Adaptive Mixtures of Local Experts" was 1991), and sparse routing in a big network goes back to Shazeer et al. in 2017, who put a layer of up to thousands of experts between LSTMs. Today many of the largest open models are MoEs:

Model	Experts	Per token	Parameters (active)
Mixtral 8x7B	8	2	47B (13B)
Qwen3-30B-A3B	128	8	30B (3B)
Qwen3-235B-A22B	128	8	235B (22B)
DeepSeek-V3	256 + 1 shared	8 + 1	671B (37B)
Qwen3-0.6B, irishQwen	dense FFN		0.6B (0.6B)

Qwen's names say it all, Qwen3-30B-A3B is `30` billion parameters with `3` billion active. irishQwen copies the small dense Qwen3, but an MoE layer could swap in for its FFN without changing a single shape.

We now have both halves of a block, attention and the FFN (or a room of experts), and the next chapter puts them together.